

TU

جامعة الطائف
TAIF UNIVERSITY

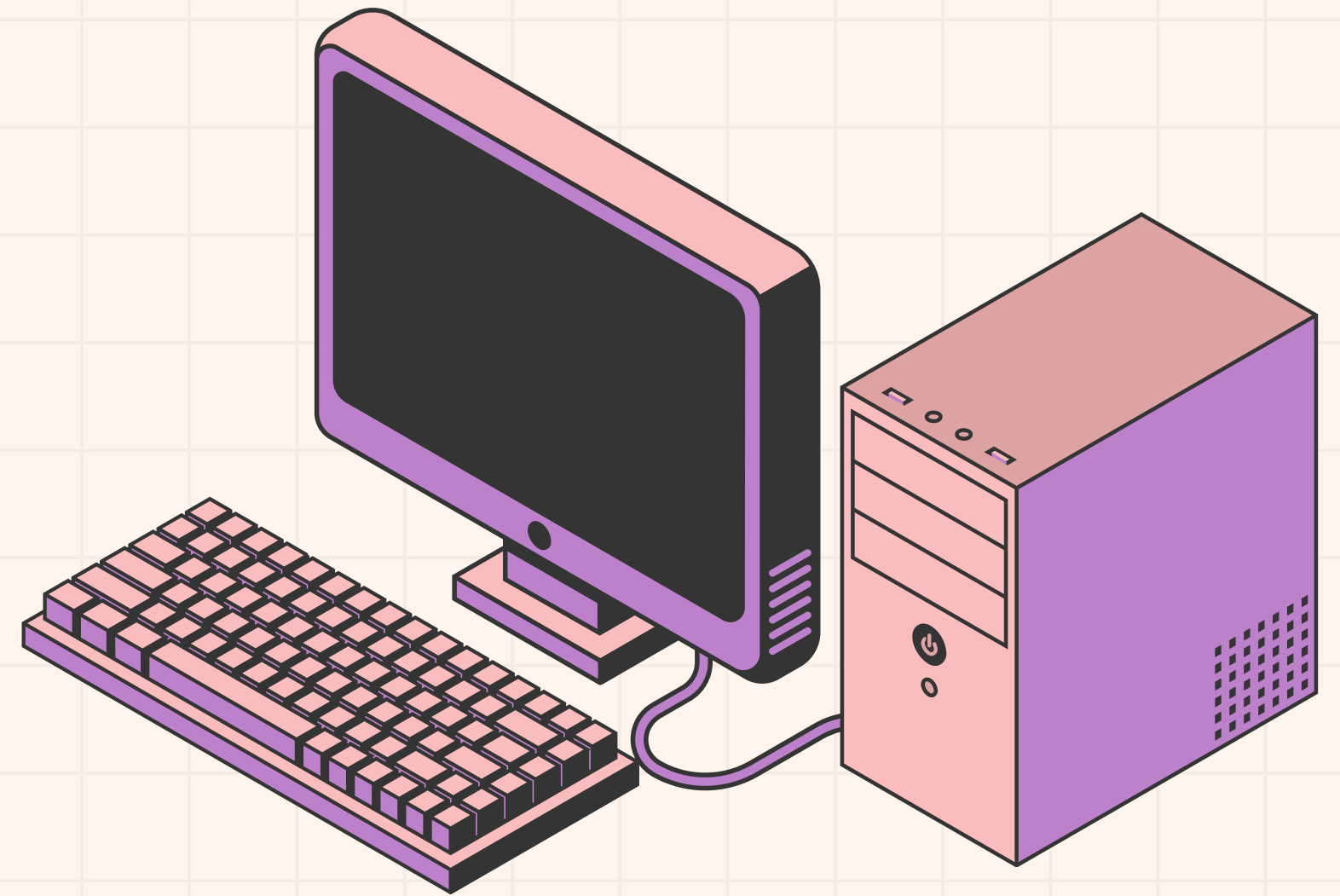
OPERATING SYSTEM

Project CPU Scheduling & Banker's Algorithm Implementation

Student Name : Sama Hassan AL-Zahrani

ID:44400233

Instructor: Dr. Sara AL-Raddadi



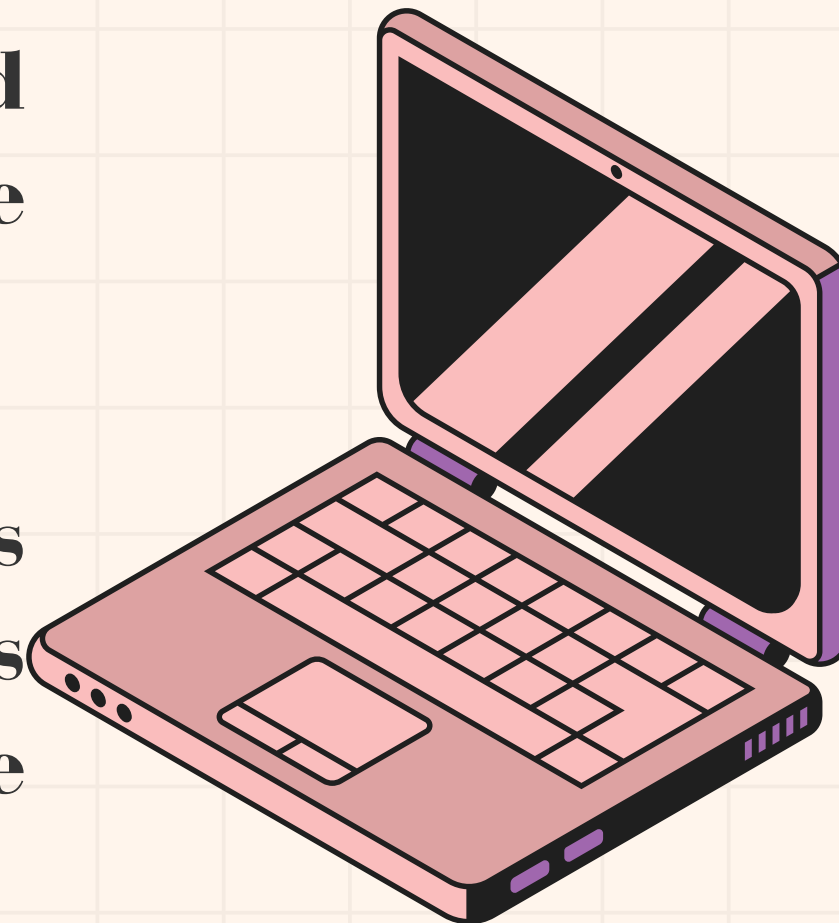
1. INTRODUCTION

Modern operating systems must handle two critical tasks efficiently:

1. deciding which process should use the CPU at any given time, and
2. allocating resources in a way that avoids deadlock.

In this project, we implemented and simulated several CPU scheduling algorithms and the Banker's Algorithm using Java. The aim is to observe how different schedulers affect waiting time and turnaround time, and how the Banker's Algorithm can be used to check whether a resource allocation state is safe or unsafe.

The implementation is fully interactive: the user enters process information and resource data via the console, and the program prints detailed tables, Gantt-like charts, averages, and the result of the Banker's safety check.



2. PROJECT OBJECTIVES

Part1- CPU Scheduling

- Implement the following
 - First-Come, First-Served (FCFS)
 - Shortest Job First (SJF – Non-Preemptive)
 - Priority Scheduling (Non-Preemptive)
 - Priority Scheduling (Preemptive)
 - Round Robin (RR)
- For each algorithm:
 - Compute **Waiting Time (WT)** and **Turnaround Time (TAT)** for every process.
 - Display the **execution order** in a Gantt-like timeline.
 - Calculate the **average WT** and **average TAT**.
- Provide a simple textual chart comparing the algorithms based on average times (bonus requirement).

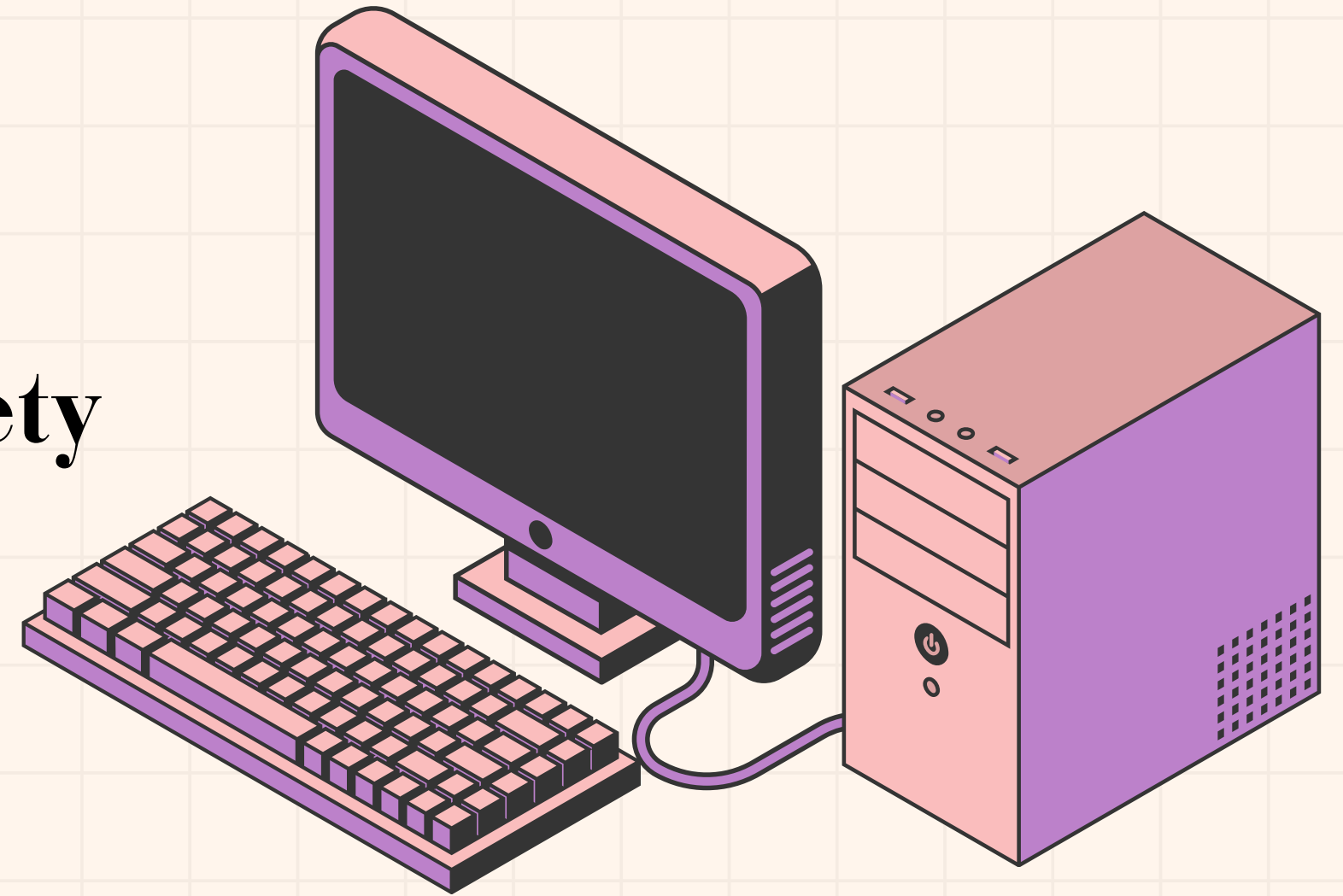
Part2- Banker's Algorithm

- Accept as input
 - Number of processes and resource types.
 - Max matrix.
 - Allocation matrix.
 - Available vector.
- Compute the **Need matrix**.
- Determine whether the current state is **SAFE** or **UNSAFE**.
- If the state is safe, print the safe sequence; otherwise, state that no **safe sequence** exists.

3. TOOLS & PROGRAMMING LANGUAGE

Programming Language: Java

- **IDE: IntelliJ IDEA / VS Code**
- **Methods Used: OOP basics, lists, queues, priority scheduling, and safety checking.**



3. CPU Scheduling Algorithms

This section briefly reviews the logic of each scheduler implemented in Java.

3.1 First-Come, First-Served (FCFS)

- Processes are queued strictly according to their **arrival time**.
- Once a process gets the CPU, it runs until completion (no preemption).
- Simple and fair in order, but long jobs can cause high waiting time for others (convoy effect).

3.3 Priority Scheduling (Non-Preemptive)

- Each process has a **priority value** (smaller number = higher priority).
- Among available processes, the one with the **highest priority** is selected.
- The running process is not interrupted until it finishes.
- Lower-priority jobs might wait for a long time if many high-priority jobs exist.

3.5 Round Robin (RR)

- Designed mainly for time-sharing and interactive systems.
- Each process receives a **time quantum** (time slice) of CPU.
- If the process is not finished within its quantum, it is placed at the end of the ready queue.
- This provides fairness among processes and good response time when the quantum is chosen appropriately.

3.2 Shortest Job First (SJF – Non-Preemptive)

- At any decision point, the scheduler selects the process with the **smallest burst time** among processes that have already arrived.
- There is no preemption; the selected process runs to completion.
- This tends to minimize average waiting time but may cause starvation for long jobs.

3.4 Priority Scheduling (Preemptive)

- Similar to priority scheduling, but **preemption is allowed**.
- If a new process arrives with a higher priority than the running process, the CPU is taken away and assigned to the higher-priority process.
- This improves responsiveness for important tasks but may increase context switching and starvation risk.

4. Input Configuration Used in the Simulation

Process	Arrival Time	Burst Time	Priority (smaller = higher)
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2
P5	6	3	5

- **Time quantum for Round Robin: $q = 4$**

These values are entered by the user at runtime. Each scheduling algorithm uses the same set of processes for a fair comparison

5.CPU Scheduling Results

5.1 FCFS Results

Programiz PRO >

Output Clear

```
=== FCFS ===
PID | WT | TAT
P1   0   8
P2   7  11
P3  10  19
P4  18  23
P5  20  23
Average WT = 11.00
Average TAT = 16.80
Gantt: P1[0->8] -> P2[8->12] -> P3[12->21] -> P4[21->26] -> P5[26->29]
```

Gantt (FCFS):

P1 [0 → 8] → P2 [8 → 12] → P3 [12 → 21] → P4 [21 → 26] → P5 [26 → 29]

- **Average Waiting Time: 11.00**
- **Average Turnaround Time: 16.80**

FCFS follows arrival order exactly. Because P1 is long, following processes suffer relatively high waiting times

5.CPU Scheduling Results

5.2 SJF (Non-Preemptive) Results

Programiz PRO >

Output Clear

```
=== SJF (Non-Preemptive) ===
PID | WT | TAT
P1   | 0   | 8
P2   | 10  | 14
P3   | 18  | 27
P4   | 12  | 17
P5   | 2   | 5
Average WT = 8.40
Average TAT = 14.20
Gantt: P1[0->8] -> P5[8->11] -> P2[11->15] -> P4[15->20] -> P3[20->29]
```

Gantt (SJF):

P1 [0 → 8] → P5 [8 → 11] → P2 [11 → 15] → P4 [15 → 20] → P3 [20 → 29]

- **Average Waiting Time: 8.40**
- **Average Turnaround Time: 14.20**

SJF significantly improves average waiting and turnaround time by running the shortest jobs first once they arrive. However, the longest job (P3) is delayed to the end.

5.CPU Scheduling Results

5.3 Priority Scheduling (Non-Preemptive) Results

Programiz PRO >

Output Clear

```
=== Priority Scheduling (Non-Preemptive) ===
PID | WT | TAT
P1   | 0   | 8
P2   | 7   | 11
P3   | 15  | 24
P4   | 9   | 14
P5   | 20  | 23
Average WT = 10.20
Average TAT = 16.00
Gantt: P1[0->8] -> P2[8->12] -> P4[12->17] -> P3[17->26] -> P5[26->29]
```

Gantt (Priority – Non-Preemptive):

P1 [0 → 8] → P2 [8 → 12] → P4 [12 → 17] → P3 [17 → 26] → P5 [26 → 29]

- **Average Waiting Time: 10.20**
- **Average Turnaround Time: 16.00**

Here, once a process starts, it is not preempted. High-priority processes (like P2 and P4) are executed earlier, while lower-priority processes like P3 and P5 experience larger waiting times.

5.CPU Scheduling Results

5.4 Priority Scheduling (Preemptive) Results

Programiz PRO >

Output Clear

```
=== Priority Scheduling (Preemptive) ===
PID | WT | TAT
P1   | 9  | 17
P2   | 0  | 4
P3   | 15 | 24
P4   | 2  | 7
P5   | 20 | 23
Average WT = 9.20
Average TAT = 15.00
Gantt: P1[0->1] -> P2[1->5] -> P4[5->10] -> P1[10->17] -> P3[17->26] -> P5[26->29]
```

Gantt (Priority – Preemptive):

P1 [0 → 1] → P2 [1 → 5] → P4 [5 → 10] → P1 [10 → 17] → P3 [17 → 26] → P5 [26 → 29]

- **Average Waiting Time: 9.20**
- **Average Turnaround Time: 15.00**

Whenever a higher-priority process appears, it interrupts the currently running one. This improves the responsiveness for P2 and P4 but slightly increases context switching.

5.CPU Scheduling Results

5.5 Round Robin (q = 4) Results

Programiz PRO >

Output Clear

```
=== Round Robin (q = 4) ===
PID | WT | TAT
P1   12  20
P2    3   7
P3   18  27
P4   20  25
P5   14  17
Average WT = 13.40
Average TAT = 19.20
Gantt: P1[0->4] -> P2[4->8] -> P3[8->12] -> P4[12->16] -> P1[16->20] -> P5[20->23] ->
        P3[23->27] -> P4[27->28] -> P3[28->29]
```

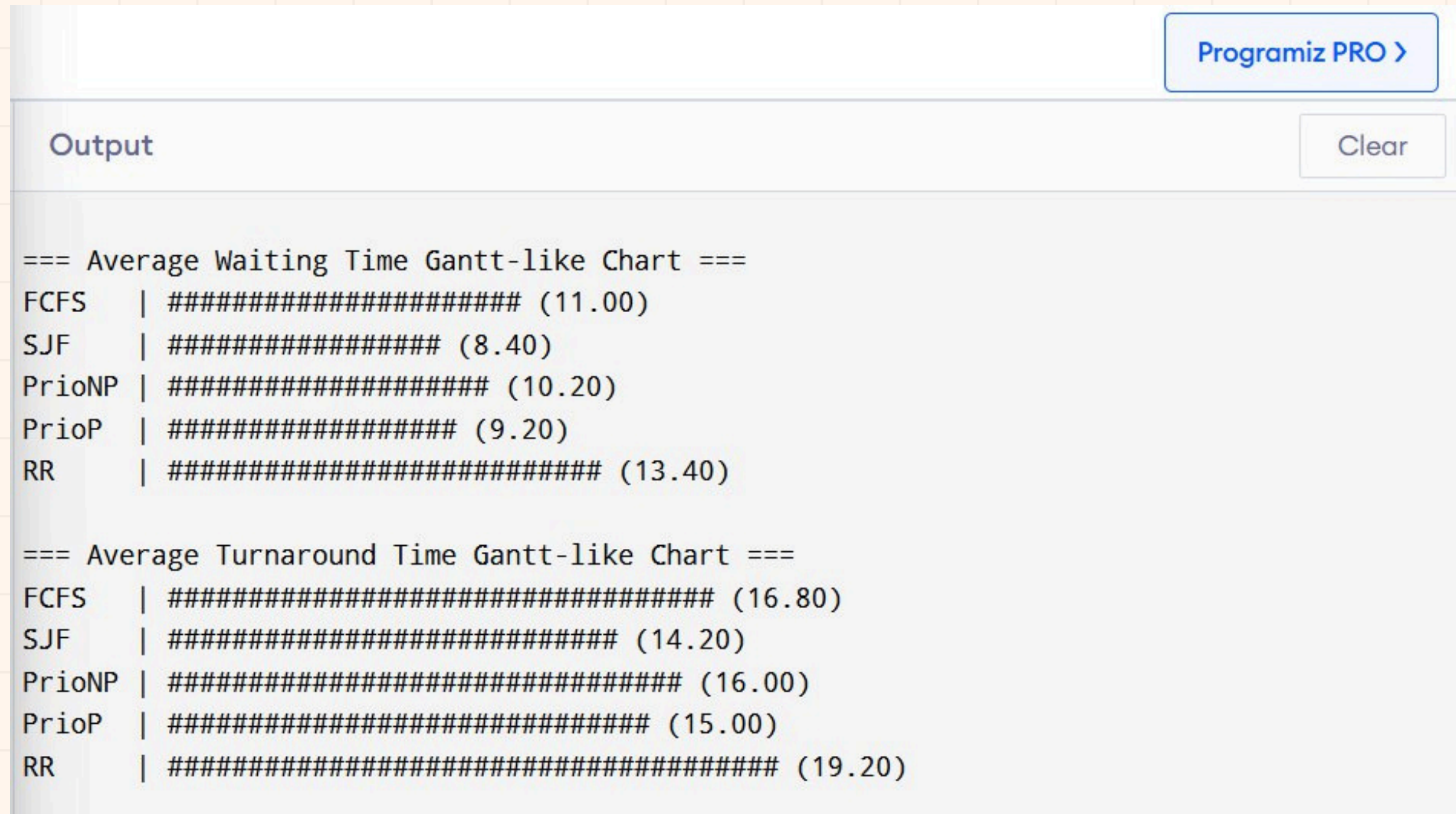
Gantt (Round Robin, q = 4):

P1 [0 → 4] → P2 [4 → 8] → P3 [8 → 12] → P4 [12 → 16] → P1 [16 → 20] → P5 [20 → 23] → P3 [23 → 27] → P4 [27 → 28] → P3 [28 → 29]

- **Average Waiting Time: 13.40**
- **Average Turnaround Time: 19.20**

Round Robin with a quantum of 4 units gives each process repeated short turns on the CPU. This guarantees fairness and frequent responses, but the increased context switching leads to higher average waiting and turnaround times compared to the other algorithms for this specific workload.

6. Comparison Charts



6.1 Average Waiting Time (smaller is better)

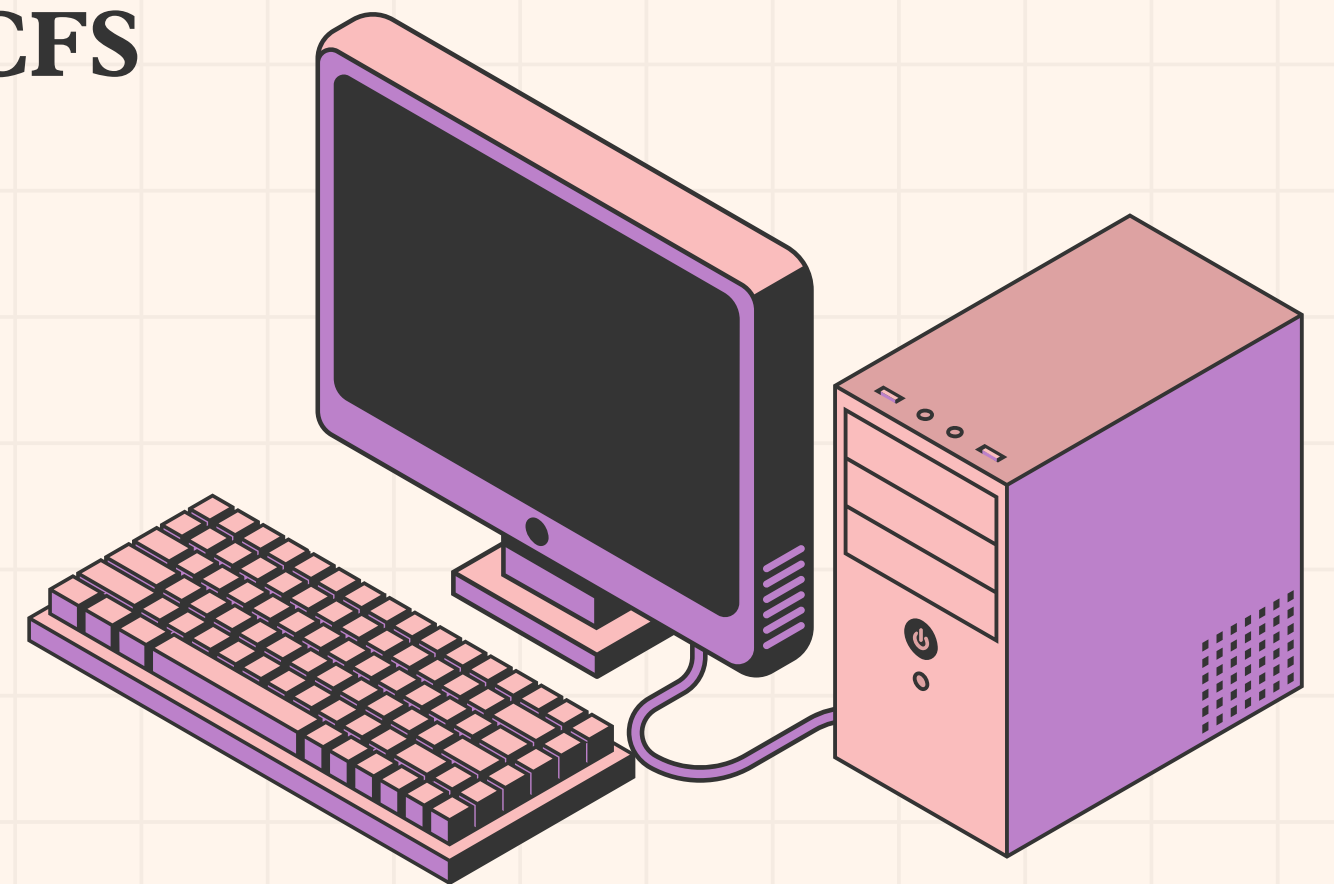
- FCFS: 11.00
- SJF: 8.40
- Priority (Non-Preemptive): 10.20
- Priority (Preemptive): 9.20
- Round Robin: 13.40 (largest WT)

6.2 Average Turnaround Time

- FCFS: 16.80
- SJF: 14.20
- Priority (Non-Preemptive): 16.00
- Priority (Preemptive): 15.00
- Round Robin: 19.20

The charts clearly show that, for this particular set of processes:

- **SJF achieves the best (lowest) average waiting and turnaround times.**
- **Round Robin is the worst in terms of averages, but it is the most fair and interactive-friendly.**
- **Priority-based algorithms lie between SJF and FCFS depending on how well priorities are assigned.**



7. Banker's Algorithm Simulation

Programiz PRO >

Output

Clear

```
=== Banker's Algorithm Simulation ===
Enter number of processes: 4
Enter number of resource types: 3
Enter Max matrix (4x3):
6 4 3
3 2 2
9 1 2
4 3 3
Enter Allocation matrix (4x3):
1 0 0
2 1 0
3 0 2
0 2 2
Enter Available vector (3 values):
2 1 1
```

7. Banker's Algorithm Simulation

7.1 Input Matrices

Number of processes: 4

Number of resource types: 3

Max matrix (4×3)

P	R1	R2	R3
P1	6	4	3
P2	3	2	2
P3	9	1	2
P4	4	3	3

Allocation matrix (4×3)

P	R1	R2	R3
P1	1	0	0
P2	2	1	0
P3	3	0	2
P4	0	2	2

Available vector

Available = (2, 1, 1)

7. Banker's Algorithm Simulation

7.2 Need Matrix

Need = Max – Allocation

Max matrix (4×3)

P	Need R1	Need R2	Need R3
P1	5	4	3
P2	1	1	2
P3	6	1	0
P4	4	1	1

Need matrix:

P1: 5 4 3

P2: 1 1 2

P3: 6 1 0

P4: 4 1 1

System state: UNSAFE (no safe sequence)

7. Banker's Algorithm Simulation

7.3 Safety Check

Initial Work = Available = (2, 1, 1).

The algorithm tries to find a process whose $\text{Need} \leq \text{Work}$:

- P1: (5,4,3) $\rightarrow$ cannot run.
- P2: (1,1,2) $\rightarrow$ last element (2) $>$ Work(3rd) = 1 $\rightarrow$ cannot run.
- P3: (6,1,0) $\rightarrow$ first element $>$ 2 $\rightarrow$ cannot run.
- P4: (4,1,1) $\rightarrow$ first element $>$ 2 $\rightarrow$ cannot run.

No process can be safely executed with the current Available vector.

7.4 Result

System state: UNSAFE (no safe sequence).

This means that, under the given Max, Allocation and Available values, the system is in a potentially deadlocked or unsafe configuration.

8. Observations and Discussion

1. **SJF** produced the lowest average waiting and turnaround times, confirming that favoring shorter jobs is very efficient when burst times are known.
2. **FCFS** is simple but suffers from the convoy effect: the long first job makes all later jobs wait longer.
3. **Priority Scheduling** allows control over which jobs are favoured:
 - In the **non-preemptive** version, once a process starts, it keeps the CPU until completion.
 - In the **preemptive** version, high-priority jobs can interrupt others and get faster service.
4. **Round Robin** gives each process fair access to the CPU and is appropriate for time-sharing systems, but the chosen quantum (4 units) leads to higher average times for this workload.
5. The **Banker's Algorithm** example shows a situation where the system is in an **UNSAFE** state: no ordering of processes guarantees that everyone can finish without running out of resources.

9. Conclusion

In this project, a complete Java program was developed for:

- Simulating five classical CPU scheduling algorithms,
- Calculating waiting and turnaround times for each process,
- Printing Gantt-like charts and comparative statistics, and
- Applying the Banker's Algorithm to judge whether a resource allocation state is safe.

Using a consistent set of inputs for Sama Hassan, we saw how each scheduling policy impacts system performance and fairness. We also confirmed that poor resource allocation can lead to an unsafe state, as detected by the Banker's Algorithm.

Overall, the project provides a practical, hands-on understanding of key operating system concepts: **CPU scheduling, resource allocation, and deadlock avoidance.**

code

Main.java



Share

Run

```
1 import java.util.*;
2
3 public class Main {
4
5     static class Job {
6         int id;
7         int arrival;
8         int burst;
9         int priority;
10        int waiting;
11        int turnaround;
12        int completion;
13        int remaining;
14
15        Job(int id, int arrival, int burst, int priority) {
16            this.id = id;
17            this.arrival = arrival;
18            this.burst = burst;
19            this.priority = priority;
20            this.remaining = burst;
```



Become a PRO

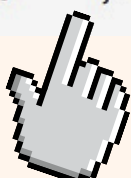
Main.java



Share

Run

```
21     }
22
23     Job copy() {
24         return new Job(id, arrival, burst, priority);
25     }
26 }
27
28 static class Result {
29     List<Job> jobs;
30     double avgWaiting;
31     double avgTurnaround;
32     String gantt;
33
34     Result(List<Job> jobs, double aw, double at, String g) {
35         this.jobs = jobs;
36         this.avgWaiting = aw;
37         this.avgTurnaround = at;
38         this.gantt = g;
39     }
40 }
```



Main.java



Share

Run

```

41
42 public static void main(String[] args) {
43     Scanner in = new Scanner(System.in);
44
45     System.out.println("Operating Systems Project - CPU Scheduling and
46         Banker's Algorithm");
47     System.out.println("Student: Sama Hassan (44400233)");
48     System.out.println();
49     System.out.print("Enter number of processes: ");
50     int n = in.nextInt();
51     List<Job> base = new ArrayList<>();
52
53     for (int i = 0; i < n; i++) {
54         int id = i + 1;
55         System.out.println("Process P" + id + ":");
56         System.out.print("Arrival time: ");
57         int at = in.nextInt();
58         System.out.print("Burst time: ");
59         int bt = in.nextInt();

```



Main.java



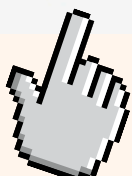
Share

Run

```

60     System.out.print("Priority (smaller = higher priority): ");
61     int pr = in.nextInt();
62     base.add(new Job(id, at, bt, pr));
63     System.out.println();
64 }
65
66 System.out.print("Enter time quantum for Round Robin: ");
67 int quantum = in.nextInt();
68 System.out.println();
69
70 Result rFdfs = fcfs(copyList(base));
71 Result rSjf = sjf(copyList(base));
72 Result rPrioNP = priorityNonPreemptive(copyList(base));
73 Result rPrioP = priorityPreemptive(copyList(base));
74 Result rRR = roundRobin(copyList(base), quantum);
75
76 printResult("FCFS", rFdfs);
77 printResult("SJF (Non-Preemptive)", rSjf);
78 printResult("Priority Scheduling (Non-Preemptive)", rPrioNP);
79 printResult("Priority Scheduling (Preemptive)", rPrioP);

```



Main.java



Share

Run

```

80     printResult("Round Robin (q = " + quantum + ")", rRR);
81
82     drawCharts(rFcfs, rSjf, rPrioNP, rPrioP, rRR);
83
84     bankersAlgorithm(in);
85 }
86
87 static List<Job> copyList(List<Job> src) {
88     List<Job> out = new ArrayList<>();
89     for (Job j : src) out.add(j.copy());
90     return out;
91 }
92
93 static void computeWTandTAT(List<Job> jobs) {
94     for (Job j : jobs) {
95         j.turnaround = j.completion - j.arrival;
96         j.waiting = j.turnaround - j.burst;
97     }
98 }
99

```



Main.java



Share

Run

```

100 static Result fcfs(List<Job> jobs) {
101     jobs.sort(Comparator.comparingInt((Job j) -> j.arrival).thenComparingInt
        (j -> j.id));
102     int time = 0;
103     StringBuilder g = new StringBuilder();
104     g.append("Gantt: ");
105
106     for (int i = 0; i < jobs.size(); i++) {
107         Job j = jobs.get(i);
108         if (time < j.arrival) time = j.arrival;
109         int start = time;
110         time += j.burst;
111         j.completion = time;
112         g.append("P").append(j.id).append("[").append(start).append("->"
            ).append(time).append("]");
113         if (i != jobs.size() - 1) g.append(" -> ");
114     }
115
116     computeWTandTAT(jobs);
117     double[] avgs = averages(jobs);

```



Main.java



Share

Run

```

118     return new Result(jobs, avgs[0], avgs[1], g.toString());
119 }
120
121 static Result sjf(List<Job> jobs) {
122     int time = 0;
123     int done = 0;
124     int n = jobs.size();
125     StringBuilder g = new StringBuilder();
126     g.append("Gantt: ");
127     boolean firstSegment = true;
128
129     int minArrival = Integer.MAX_VALUE;
130     for (Job j : jobs) if (j.arrival < minArrival) minArrival = j.arrival;
131     time = minArrival;
132
133     boolean[] finished = new boolean[n];
134
135     while (done < n) {
136         Job pick = null;
137         int idx = -1;

```



Main.java



Share

Run

```

137     int idx = -1;
138     for (int i = 0; i < n; i++) {
139         Job j = jobs.get(i);
140         if (!finished[i] && j.arrival <= time) {
141             if (pick == null || j.burst < pick.burst ||
142                 (j.burst == pick.burst && j.arrival < pick.arrival))
143                 {
144                     pick = j;
145                     idx = i;
146                 }
147         }
148         if (pick == null) {
149             time++;
150             continue;
151         }
152         int start = time;
153         time += pick.burst;
154         pick.completion = time;
155         finished[idx] = true;

```



Main.java



Share

Run

```
156         done++;
157
158         if (!firstSegment) g.append(" -> ");
159         g.append("P").append(pick.id).append("[").append(start).append("->"
160             ).append(time).append("]");
161         firstSegment = false;
162     }
163
164     computeWTandTAT(jobs);
165     double[] avgs = averages(jobs);
166     return new Result(jobs, avgs[0], avgs[1], g.toString());
167 }
168
169 static Result priorityNonPreemptive(List<Job> jobs) {
170     int time = 0;
171     int done = 0;
172     int n = jobs.size();
173     StringBuilder g = new StringBuilder();
174     g.append("Gantt: ");
175     boolean firstSegment = true;
```



Programiz Online Java Compiler

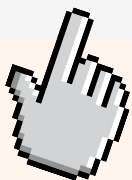
Main.java



Share

Run

```
176     int minArrival = Integer.MAX_VALUE;
177     for (Job j : jobs) if (j.arrival < minArrival) minArrival = j.arrival;
178     time = minArrival;
179
180     boolean[] finished = new boolean[n];
181
182     while (done < n) {
183         Job pick = null;
184         int idx = -1;
185         for (int i = 0; i < n; i++) {
186             Job j = jobs.get(i);
187             if (!finished[i] && j.arrival <= time) {
188                 if (pick == null || j.priority < pick.priority ||
189                     (j.priority == pick.priority && j.arrival < pick
190                         .arrival)) {
191                     pick = j;
192                     idx = i;
193                 }
194             }
195         }
```



Main.java



Run

```

195 -         if (pick == null) {
196             time++;
197             continue;
198         }
199         int start = time;
200         time += pick.burst;
201         pick.completion = time;
202         finished[idx] = true;
203         done++;
204
205         if (!firstSegment) g.append(" -> ");
206         g.append("P").append(pick.id).append("[").append(start).append("->")
207             .append(time).append("]");
208         firstSegment = false;
209     }
210
211     computeWTandTAT(jobs);
212     double[] avgs = averages(jobs);
213     return new Result(jobs, avgs[0], avgs[1], g.toString());
214 }

```



Main.java



Run

```

215 - static Result priorityPreemptive(List<Job> jobs) {
216     int n = jobs.size();
217     int time = 0;
218     int completed = 0;
219     StringBuilder g = new StringBuilder();
220     g.append("Gantt: ");
221
222     int minArrival = Integer.MAX_VALUE;
223     for (Job j : jobs) if (j.arrival < minArrival) minArrival = j.arrival;
224     time = minArrival;
225
226     for (Job j : jobs) j.remaining = j.burst;
227
228     Job current = null;
229     int segmentStart = time;
230     boolean firstSegment = true;
231
232 -     while (completed < n) {
233         Job pick = null;
234 -         for (Job j : jobs) {

```



Main.java



Share

Run

```

235 -         if (j.arrival <= time && j.remaining > 0) {
236 -             if (pick == null || j.priority < pick.priority ||
237 -                 (j.priority == pick.priority && j.arrival < pick
                     .arrival)) {
238 -                 pick = j;
239 -             }
240 -         }
241 -     }
242 -
243 -     if (pick == null) {
244 -         time++;
245 -         continue;
246 -     }
247 -
248 -     if (current == null || current.id != pick.id) {
249 -         if (current != null) {
250 -             if (!firstSegment) g.append(" -> ");
251 -             g.append("P").append(current.id)
252 -                 .append("[").append(segmentStart).append("->").append
                     (time).append("]");

```



Programiz Online Java Compiler

Main.java



Share

Run

```

253         firstSegment = false;
254     }
255     current = pick;
256     segmentStart = time;
257 }
258
259 pick.remaining--;
260 time++;
261
262 -     if (pick.remaining == 0) {
263         pick.completion = time;
264         completed++;
265 -         if (current != null && current.id == pick.id) {
266             if (!firstSegment) g.append(" -> ");
267             g.append("P").append(current.id)
268                 .append("[").append(segmentStart).append("->").append
                     (time).append("]");
269             firstSegment = false;
270             current = null;
271         }

```



Main.java



Run

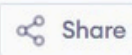
```

272     }
273 }
274
275     computeWTandTAT(jobs);
276     double[] avgs = averages(jobs);
277     return new Result(jobs, avgs[0], avgs[1], g.toString());
278 }
279
280 static Result roundRobin(List<Job> jobs, int quantum) {
281     int n = jobs.size();
282     for (Job j : jobs) j.remaining = j.burst;
283
284     int time = 0;
285     int completed = 0;
286     StringBuilder g = new StringBuilder();
287     g.append("Gantt: ");
288     boolean firstSegment = true;
289
290     int minArrival = Integer.MAX_VALUE;
291     for (Job j : jobs) if (j.arrival < minArrival) minArrival = j.arrival;

```



Main.java



Run

```

292     time = minArrival;
293
294     Queue<Job> q = new ArrayDeque<>();
295     boolean[] inQueue = new boolean[n];
296
297     while (completed < n) {
298         for (int i = 0; i < n; i++) {
299             Job j = jobs.get(i);
300             if (!inQueue[i] && j.arrival <= time && j.remaining > 0) {
301                 q.add(j);
302                 inQueue[i] = true;
303             }
304         }
305
306         if (q.isEmpty()) {
307             time++;
308             continue;
309         }
310
311         Job cur = q.poll();

```



Main.java



Share

Run

```

312         int idx = cur.id - 1;
313         int exec = Math.min(quantum, cur.remaining);
314         int start = time;
315         time += exec;
316         cur.remaining -= exec;
317
318         if (!firstSegment) g.append(" -> ");
319         g.append("P").append(cur.id).append("[").append(start).append("->")
320           .append(time).append("]");
321         firstSegment = false;
322
323         for (int i = 0; i < n; i++) {
324             Job j = jobs.get(i);
325             if (!inQueue[i] && j.arrival <= time && j.remaining > 0) {
326                 q.add(j);
327                 inQueue[i] = true;
328             }
329
330             if (cur.remaining > 0) {

```



Main.java



Share

Run

```

331         q.add(cur);
332     } else {
333         cur.completion = time;
334         completed++;
335     }
336 }
337
338 computeWTandTAT(jobs);
339 double[] avgs = averages(jobs);
340 return new Result(jobs, avgs[0], avgs[1], g.toString());
341 }
342
343 static double[] averages(List<Job> jobs) {
344     double sumW = 0;
345     double sumT = 0;
346     for (Job j : jobs) {
347         sumW += j.waiting;
348         sumT += j.turnaround;
349     }
350     return new double[]{sumW / jobs.size(), sumT / jobs.size()};

```



Main.java



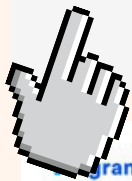
Share

Run

```

351     }
352
353     static void printResult(String title, Result r) {
354         System.out.println("=== " + title + " ===");
355         System.out.println("PID | WT | TAT");
356         for (Job j : r.jobs) {
357             System.out.printf("P%-2d %3d %4d%n", j.id, j.waiting, j.turnaround);
358         }
359         System.out.printf("Average WT = %.2f%n", r.avgWaiting);
360         System.out.printf("Average TAT = %.2f%n", r.avgTurnaround);
361         System.out.println(r.gantt);
362         System.out.println();
363     }
364
365     static void drawCharts(Result fcfs, Result sjf, Result prioNP, Result prioP,
        Result rr) {
366         System.out.println("=== Average Waiting Time Gantt-like Chart ===");
367         drawBar("FCFS ", fcfs.avgWaiting);
368         drawBar("SJF ", sjf.avgWaiting);
369         drawBar("PrioNP", prioNP.avgWaiting);

```



Main.java



Share

Run

```

370         drawBar("PrioP ", prioP.avgWaiting);
371         drawBar("RR ", rr.avgWaiting);
372         System.out.println();
373
374         System.out.println("=== Average Turnaround Time Gantt-like Chart ===");
375         drawBar("FCFS ", fcfs.avgTurnaround);
376         drawBar("SJF ", sjf.avgTurnaround);
377         drawBar("PrioNP", prioNP.avgTurnaround);
378         drawBar("PrioP ", prioP.avgTurnaround);
379         drawBar("RR ", rr.avgTurnaround);
380         System.out.println();
381     }
382
383     static void drawBar(String name, double value) {
384         int len = (int) Math.round(value * 2);
385         StringBuilder sb = new StringBuilder();
386         for (int i = 0; i < len; i++) sb.append('#');
387         System.out.printf("%s | %s (%.2f)%n", name, sb.toString(), value);
388     }
389

```



Main.java



Share

Run

```

391 System.out.println("=== Banker's Algorithm Simulation ===");
392 System.out.print("Enter number of processes: ");
393 int n = in.nextInt();
394 System.out.print("Enter number of resource types: ");
395 int m = in.nextInt();
396
397 int[][] max = new int[n][m];
398 int[][] alloc = new int[n][m];
399 int[][] need = new int[n][m];
400 int[] avail = new int[m];
401
402 System.out.println("Enter Max matrix (" + n + "x" + m + "):");
403 for (int i = 0; i < n; i++)
404     for (int j = 0; j < m; j++)
405         max[i][j] = in.nextInt();
406
407 System.out.println("Enter Allocation matrix (" + n + "x" + m + "):");
408 for (int i = 0; i < n; i++)
409     for (int j = 0; j < m; j++)
410         alloc[i][j] = in.nextInt();

```



Main.java



Share

Run

```

412 System.out.println("Enter Available vector (" + m + " values):");
413 for (int j = 0; j < m; j++)
414     avail[j] = in.nextInt();
415
416 for (int i = 0; i < n; i++)
417     for (int j = 0; j < m; j++)
418         need[i][j] = max[i][j] - alloc[i][j];
419
420 System.out.println("Need matrix:");
421 for (int i = 0; i < n; i++) {
422     System.out.print("P" + (i + 1) + ": ");
423     for (int j = 0; j < m; j++)
424         System.out.print(need[i][j] + " ");
425     System.out.println();
426 }
427
428 boolean[] finished = new boolean[n];
429 List<Integer> safeSeq = new ArrayList<>();
430 int[] work = Arrays.copyOf(avail, m);
431 boolean progress:

```



Main.java



Share

Run

```

431     boolean progress;
432
433     while (safeSeq.size() < n) {
434         progress = false;
435         for (int i = 0; i < n; i++) {
436             if (finished[i]) continue;
437             boolean can = true;
438             for (int j = 0; j < m; j++) {
439                 if (need[i][j] > work[j]) {
440                     can = false;
441                     break;
442                 }
443             }
444             if (can) {
445                 for (int j = 0; j < m; j++)
446                     work[j] += alloc[i][j];
447                 finished[i] = true;
448                 safeSeq.add(i);
449                 progress = true;
450             }

```



Main.java



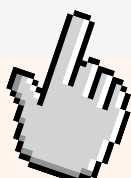
Share

Run

```

450     }
451 }
452 if (!progress) break;
453 }
454
455 boolean safe = safeSeq.size() == n;
456 if (!safe) {
457     System.out.println("System state: UNSAFE (no safe sequence)");
458 } else {
459     System.out.println("System state: SAFE");
460     System.out.print("Safe sequence: ");
461     for (int i = 0; i < safeSeq.size(); i++) {
462         System.out.print("P" + (safeSeq.get(i) + 1));
463         if (i != safeSeq.size() - 1) System.out.print(" -> ");
464     }
465     System.out.println();
466 }
467 }
468 }
469

```



output

<

>

Operating Systems Project - CPU Scheduling and Banker's Algorithm

Student: Sama Hassan (44400233)

Enter number of processes: 5

Process P1:

Arrival time: 0

Burst time: 8

Priority (smaller = higher priority): 3

Process P2:

Arrival time: 1

Burst time: 4

Priority (smaller = higher priority): 1

Process P3:

Arrival time: 2

Burst time: 9

Priority (smaller = higher priority): 4

Process P4:

⊗ □ —

Output

Clear

Process P4:

Arrival time: 3

Burst time: 5

Priority (smaller = higher priority): 2

Process P5:

Arrival time: 6

Burst time: 3

Priority (smaller = higher priority): 5

Enter time quantum for Round Robin: 4

⊗ □ —

Output

Clear

=== FCFS ===

PID	WT	TAT
P1	0	8
P2	7	11
P3	10	19
P4	18	23
P5	20	23

Average WT = 11.00

Average TAT = 16.80

Gantt: P1[0->8] -> P2[8->12] -> P3[12->21] -> P4[21->26] -> P5[26->29]

Output

Clear

```

=== SJF (Non-Preemptive) ===
PID | WT | TAT
P1   0   8
P2  10  14
P3  18  27
P4  12  17
P5   2   5
Average WT = 8.40
Average TAT = 14.20
Gantt: P1[0->8] -> P5[8->11] -> P2[11->15] -> P4[15->20] -> P3[20->29]

```

```

=== Priority Scheduling (Non-Preemptive) ===
PID | WT | TAT
P1   0   8
P2   7  11
P3  15  24
P4   9  14
P5  20  23
Average WT = 10.20
Average TAT = 16.00
Gantt: P1[0->8] -> P2[8->12] -> P4[12->17] -> P3[17->26] -> P5[26->29]

```

Output

Clear

```

=== Round Robin (q = 4) ===
PID | WT | TAT
P1  12  20
P2   3   7
P3  18  27
P4  20  25
P5  14  17
Average WT = 13.40
Average TAT = 19.20
Gantt: P1[0->4] -> P2[4->8] -> P3[8->12] -> P4[12->16] -> P1[16->20] -> P5[20->23] ->
      P3[23->27] -> P4[27->28] -> P3[28->29]

```

Output

Clear

```

=== Priority Scheduling (Preemptive) ===
PID | WT | TAT
P1   9  17
P2   0   4
P3  15  24
P4   2   7
P5  20  23
Average WT = 9.20
Average TAT = 15.00
Gantt: P1[0->1] -> P2[1->5] -> P4[5->10] -> P1[10->17] -> P3[17->26] -> P5[26->29]

```



```
Output

=== Average Waiting Time Gantt-like Chart ===
FCFS | ##### (11.00)
SJF  | ##### (8.40)
PrioNP | ##### (10.20)
PrioP | ##### (9.20)
RR   | ##### (13.40)

=== Average Turnaround Time Gantt-like Chart ===
FCFS | ##### (16.80)
SJF  | ##### (14.20)
PrioNP | ##### (16.00)
PrioP | ##### (15.00)
RR   | ##### (19.20)
```

```
Output

=== Banker's Algorithm Simulation ===
Enter number of processes: 4
Enter number of resource types: 3
Enter Max matrix (4x3):
6 4 3
3 2 2
9 1 2
4 3 3
Enter Allocation matrix (4x3):
1 0 0
2 1 0
3 0 2
0 2 2
Enter Available vector (3 values):
2 1 1
```

```
⊗ □ _

Need matrix:
P1: 5 4 3
P2: 1 1 2
P3: 6 1 0
P4: 4 1 1
System state: UNSAFE (no safe sequence)
```